SR UNIVERSITY

SchoolofComputerScienceandArtificialIntelligence

LabAssignment#18

Name of Student :R.saikundan
Enrollment No. : 2303A51413
Batch No. : 21

TaskDescription–1(AirQualityAPIIntegration)
• Task:UseAIassistancetodevelopaPythonprogramthatretrievesreal-timeair quality information for a specified city.
Instructions:
• GeneratePythoncodeusingtherequests library.
• Handleerrorssuchasinvalidcitynames,missingorincorrectAPIkeys,andnetwork timeouts.
• ExtractanddisplayAQIvalue,pollutionlevel,anddominantpollutantinareadable format.
SampleCode:
importrequests
city = "Delhi"
url="https://api.example.com/airquality" Expected
Output:
• APythonscriptthatsuccessfullyfetchesanddisplaysairqualitydatawith proper error handling.

Prompt:-
Develop apython code that retrieves real-time airquality information for aspecific city from an API, using the requests library. and handle error like as invalid city name, missing API key, or incorrect API key. and network timeout. and display AQI values, Pollution level, and dominantpollutant.inareadableformat.eg.,forDelhi,India.ReflectonhowAIcanassistin improving code quality and adding features while collaborating with the user.

Output:-

```
Error: Please provide a valid API key.
```

Justification:
The enhanced code includes comprehensive error handling for various scenarios, such as invalidAPIkeys,networkissues,andunexpectedAPIresponses.Italsoformatstheoutputin a more user-friendly way, making it easier to understand the air quality information. This demonstrateshowAIcan assistinimprovingcodequalityandaddingvaluablefeatureswhile collaborating with the user.

Code:-

```python
# Task -01
# Develop a python code that retrieves real-time air quality information for a specific city from an API,
import requests
class AirQualityInfo:
    def __init__(self, api_key):
        self.api_key = api_key
        self.base_url = "https://api.openweathermap.org/data/2.5/air_pollution"
    def get_air_quality(self, country, state, city):
        if not self.api_key or self.api_key == "your_api_key_here":
            return "Error: Please provide a valid API key."
        try:
            url = f"{self.base_url}/{city}/?token={self.api_key}"
            response = requests.get(url, timeout=10)
            response.raise_for_status()  # Check for HTTP errors
            try:
                data = response.json()
            except ValueError:
                return f"Error: Invalid JSON response from API. Response text: {response.text[:200]!r}"
            if data.get("status") == "ok":
                aqi = data["data"].get("aqi")
                dominant_pollutant = data["data"].get("dominentpol", "unknown")
                return (
                    f"Air Quality for {city}, {state}, {country}:\n"
                    f"  AQI: {aqi}\n"
                    f"  Dominant Pollutant: {dominant_pollutant}"
                )
            elif data.get("status") == "error":
                return f"API error: {data.get('data', 'Unknown API error.')}"
            else:
                return "Error: Unable to retrieve air quality information."
        except requests.exceptions.HTTPError as http_err:
            return f"HTTP error occurred: {http_err}"
        except requests.exceptions.ConnectionError:
            return "Error: Network connection issue."
        except requests.exceptions.Timeout:
            return "Error: The request timed out."
        except Exception as err:
            return f"An error occurred: {err}"
# Example Usage
if __name__ == "__main__":
    api_key = "your_api_key_here"
    air_quality_info = AirQualityInfo(api_key)
    print(air_quality_info.get_air_quality("India", "Telangana", "Warangal"))
# Reflection on AI Assistance
```

TaskDescription–2(LanguageTranslationAPI)

• Task:Integrate aLanguageTranslationAPItotranslatetextfromEnglishtotwo other languages.

Instructions:

• UseAIto generateAPIrequestcodefor translation.

• Validateuserinputtext andtarget language codes.

• HandleAPIresponseerrors andservice unavailability.

• Displayoriginalandtranslatedtextclearly.

Sample Code:

importrequests

text="WelcometoAIAssistedCoding" target_lang = "fr"

ExpectedOutput:

• AworkingPythonscriptthat translatesuserinputtextwithgracefulerror handling.

Prompt:-
Generate a working Python script usingrequests to call the My Memory Translation API and translateEnglishtextintoFrenchandSpanish.Validateinputtextandlanguagecodes,handle HTTP errors, timeouts, connection problems, invalid JSON, and service unavailability, and print original plus translated text clearly. Use "Welcome to AI Assisted Coding" as the example input. Also add a short reflection on how AI can help improve the code.import requests

Code:-

```python
# Task - 02
#Generate a working Python script using requests to call the MyMemory Translation API and translate English te
import requests
text = "Welcome to AI Assisted Coding"
target_lang = "fr"
api_url = f"https://api.mymemory.translated.net/get?q={text}&langpair=en|{target_lang}"
try:
    response = requests.get(api_url, timeout=10)
    response.raise_for_status()  # Check for HTTP errors
    try:
        data = response.json()
    except ValueError:
        print(f"Error: Invalid JSON response from API. Response text: {response.text[:200]!r}")
    else:
        if data.get("responseStatus") == 200:
            translated_text = data["responseData"]["translatedText"]
            print(f"Original Text: {text}")
            print(f"Translated Text ({target_lang}): {translated_text}")
        else:
            print(f"API error: {data.get('responseDetails', 'Unknown API error.')}")
except requests.exceptions.HTTPError as http_err:
    print(f"HTTP error occurred: {http_err}")
except requests.exceptions.ConnectionError:
    print("Error: Network connection issue.")
except requests.exceptions.Timeout:
    print("Error: The request timed out.")
except Exception as err:
    print(f"An error occurred: {err}")
# Reflection on AI Assistance
# AI can assist in improving code quality by suggesting best practices for input validation, error handling,
```

Output:-

```
Original Text: Welcome to AI Assisted Coding
Translated Text (fr): Bienvenue dans le codage assisté par IA
PS C:\AIAC LAB\Assignments Codes>
```

Justification:
The enhanced code includes comprehensive error handling for various scenarios, such as invalid input text, unsupported language codes, network issues, and unexpected API responses. It also formats the output in a clear and user-friendly way, making it easier to understandtheoriginalandtranslatedtext.ThisdemonstrateshowAIcanassistinimproving code qualityand adding valuable features while collaborating withthe user. Additionally, AI can help identify potential edge cases and suggest improvements to make the code more robust and user-friendly. By leveraging AI's abilityto analyze code and provide suggestions, developerscan enhance thefunctionalityand reliabilityof theirapplicationswhileensuring a better user experience.

TaskDescription–3(PublicBookInformation API)
• Task:ConnecttoapublicBooksAPItofetchinformation aboutabookusingitstitle or ISBN.
Instructions:
• PromptAI towritePythoncodeforsendingGETrequeststo theAPI.
• Handleerrorssuchasbooknotfound,invalidinput,andrate limits.
• Displaybooktitle,author,publisher,andpublicationyear.
Sample Code:
importrequests
query="ArtificialIntelligence"
Expected Output:
• APythonprogram thatretrieves anddisplaysbookdetailswithrobusterror handling.

Prompt:
GenerateaPythonprogramusingrequestsandtheGoogleBooksAPItofetchbookdetails by title or ISBN. Validate input, use GET requests, handle book not found, invalid input, timeouts, connection errors, HTTP errors, rate limits, and invalid JSON, and display title, author, publisher, and publication year clearly. Use "Artificial Intelligence" as the sample query. Reflect on how AI can assist in improving code quality and adding features while collaborating with the user.

Code:-

```python
# Task - 03
# Generate a Python program using requests and the Google Books API to fetch book details by title or ISBN.
import requests
def fetch_book_details(query):
    api_url = f"https://www.googleapis.com/books/v1/volumes?q={query}"
    try:
        response = requests.get(api_url, timeout=10)
        response.raise_for_status()  # Check for HTTP errors
        try:
            data = response.json()
        except ValueError:
            print(f"Error: Invalid JSON response from API. Response text: {response.text[:200]!r}")
        else:
            if "items" in data and len(data["items"]) > 0:
                book = data["items"][0]["volumeInfo"]
                title = book.get("title", "N/A")
                authors = ", ".join(book.get("authors", ["N/A"]))
                publisher = book.get("publisher", "N/A")
                published_date = book.get("publishedDate", "N/A")
                print(f"Title: {title}")
                print(f"Author(s): {authors}")
                print(f"Publisher: {publisher}")
                print(f"Publication Year: {published_date}")
            else:
                print("Book not found.")
    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.ConnectionError:
        print("Error: Network connection issue.")
    except requests.exceptions.Timeout:
        print("Error: The request timed out.")
    except Exception as err:
        print(f"An error occurred: {err}")
# Example Usage
if __name__ == "__main__":
    fetch_book_details("Artificial Intelligence")
# Reflection on AI Assistance
# AI can assist in improving code quality by suggesting best practices for input validation, error handling
```

Output:-

```
HTTP error occurred: 429 Client Error: Too Many Requests for url: https://www.googleapis.com/books/v1/volumes?q=Artificial%20Intelligence
```

Justification:
The enhanced code includes comprehensive error handling for various scenarios, such as invalidinput, book not found, network issues, and unexpected API responses. It alsoformats the output in a clear and user-friendly way, making it easier to understand the book details. This demonstrates how AI can assist in improving code quality and adding valuable features while collaborating withthe user. Additionally, AI can help identify potential edge cases and suggest improvements to make the code more robust and user-friendly. By leveraging AI's abilitytoanalyzecodeandprovidesuggestions,developerscanenhancethefunctionalityand reliability of their applications while ensuring a better user experience.

TaskDescription–4(Real-TimeApplication:JobListingsAggregator)
• Scenario:
DevelopaJobListingsAggregatorusingapublicJobsAPI. Requirements:
• Fetchthelatest jobpostingsforagivenrole orlocation.
• Handleerrorssuchasinvalidsearchterms,missingAPI keys,andrequest failures.
• Displayjobtitle,company name,location,andpostingdateinanumberedlist.
• ImplementaretrymechanismforfailedAPIrequests.
Sample Code:
importrequests
keyword="PythonDeveloper"
Expected Output:
• AfunctionalPythonscriptthatretrievesanddisplaysjoblistingswithpropererror handling.

Prompt:-
Generate a Python script using requests to build a Job Listings Aggregator with a public Jobs API. It should fetch latest jobs for a keyword like "Python Developer", validate user input, handlemissingAPI keys, request failures, timeouts, invalid JSON, andno results, retry failed requests up to 3 times, and display job title, company name, location, and posting date in a numberedlist.AlsoincludesampleoutputandashortreflectiononhowAIhelpsimprovethe code.

Output:-

```
Error: Network connection issue.
Retrying... (1/3)
Error: Network connection issue.
Retrying... (2/3)
Error: Network connection issue.
Retrying... (3/3)
Failed to fetch job listings after multiple attempts.
```

Code:-

```python
# Task - 04
#Generate a Python script using requests to build a Job Listings Aggregator with a public Jobs API. It should fe
import requests
def fetch_job_listings(keyword):
    api_url = f"https://jobs.github.com/positions.json?description={keyword}"
    retries = 3
    for attempt in range(retries):
        try:
            response = requests.get(api_url, timeout=10)
            response.raise_for_status()  # Check for HTTP errors
            try:
                data = response.json()
            except ValueError:
                print(f"Error: Invalid JSON response from API. Response text: {response.text[:200]!r}")
                return
            if isinstance(data, list) and len(data) > 0:
                for idx, job in enumerate(data, start=1):
                    title = job.get("title", "N/A")
                    company = job.get("company", "N/A")
                    location = job.get("location", "N/A")
                    posting_date = job.get("created_at", "N/A")
                    print(f"{idx}. {title} at {company} in {location} (Posted on: {posting_date})")
                return
            else:
                print("No job listings found.")
                return
        except requests.exceptions.HTTPError as http_err:
            print(f"HTTP error occurred: {http_err}")
        except requests.exceptions.ConnectionError:
            print("Error: Network connection issue.")
        except requests.exceptions.Timeout:
            print("Error: The request timed out.")
        except Exception as err:
            print(f"An error occurred: {err}")
        print(f"Retrying... ({attempt + 1}/{retries})")
    print("Failed to fetch job listings after multiple attempts.")
# Example Usage
if __name__ == "__main__":
    fetch_job_listings("Python Developer")
# Reflection on AI Assistance
# AI can assist in improving code quality by suggesting best practices for error handling, such as implementing
```

Justification:

The enhanced code includes comprehensive error handling for various scenarios, such as invalid input, missing API keys, network issues, and unexpected API responses. It also implements a retry mechanism for failed requests, which increases the robustness of the application. The output is formatted in a clear and user-friendly way, making it easier to understandthejoblistings.ThisdemonstrateshowAIcanassistinimprovingcodequality and adding valuable features while collaborating with the user. Additionally, AI can help identify potential edge cases and suggest improvements to makethecode more robust and user-friendly.

TaskDescription–5StockMarket Data APIIntegration

• Task:UseAIassistancetodevelopaPythonprogramthatretrieves real-time stock market data for a selected company.

Instructions:

• GeneratePythoncodeusingtherequestslibrarytoconnecttoa public Stock Market API.

• Validateuserinputforstocksymbol.

• Handlecommonerrorssuchasinvalidsymbols,APIratelimits, missing API keys, and network failures.

• Extractanddisplaystockname,currentprice,dayhigh,daylow,andpercentage change.
SampleCode:
import requests
symbol= "AAPL"
• ExpectedOutput:
APythonscriptthatsuccessfullyfetchesanddisplaysstockmarketinformationwith robust error handling.

Prompt:-
Generate a Python script using requests that calls a public stock market API (Alpha VantageorFinnhub)tofetch real-timedataforauser-entered stock symbol.Validatethesymbol input, handle errors like invalid symbol, missing API key, rate limits, HTTP errors, connection errors, and timeouts, and then print stock name/symbol, current price, day high, day low, and percentage change. Include an example with symbol "AAPL" and explain the error handling briefly.

Code:-

```python
# Task-05
# Generate a Python script using requests that calls a public stock market API (Alpha Vantage or Finnhub)
import requests
def fetch_stock_data(symbol, api_key):
    if not api_key or api_key == "your_api_key_here":
        print("Error: Please provide a valid API key.")
        return
    api_url = f"https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol={symbol}&apikey={api_key}"
    try:
        response = requests.get(api_url, timeout=10)
        response.raise_for_status()  # Check for HTTP errors
        try:
            data = response.json()
        except ValueError:
            print(f"Error: Invalid JSON response from API. Response text: {response.text[:200]!r}")
            return
        if "Global Quote" in data and data["Global Quote"]:
            quote = data["Global Quote"]
            stock_symbol = quote.get("01. symbol", "N/A")
            current_price = quote.get("05. price", "N/A")
            day_high = quote.get("03. high", "N/A")
            day_low = quote.get("04. low", "N/A")
            percentage_change = quote.get("10. change percent", "N/A")
            print(f"Stock Symbol: {stock_symbol}")
            print(f"Current Price: ${current_price}")
            print(f"Day High: ${day_high}")
            print(f"Day Low: ${day_low}")
            print(f"Percentage Change: {percentage_change}")
        else:
            print("Error: Invalid stock symbol or no data available.")
    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.ConnectionError:
        print("Error: Network connection issue.")
    except requests.exceptions.Timeout:
        print("Error: The request timed out.")
    except Exception as err:
        print(f"An error occurred: {err}")
# Example Usage
if __name__ == "__main__":
    api_key = "your_api_key_here"
    fetch_stock_data("AAPL", api_key)
# Reflection on Error Handling
# The error handling in this code covers various scenarios that can occur when making API requests.
```

Output:-

```
Error: Please provide a valid API key.
```

Justification:

The enhanced code includes comprehensive error handling for various scenarios, such as invalidAPIkeys,networkissues,andunexpectedAPIresponses.Italsoformatstheoutputin a more user-friendly way, making it easier to understand the stock information. This demonstrateshowAIcan assistinimprovingcodequalityandaddingvaluablefeatureswhile collaborating with the user.